
UNIT 3**ELEMENTS OF DOT NET LANGUAGES****[4hrs]****3.1. Introduction and Data types, identifiers, variables and constants****3.1.1. Introduction:**

C# is part of .Net framework and is used for writing .Net applications.

3.1.1.1. The .Net Framework: The .Net framework is a revolutionary platform that helps us to write the following types of applications:

- ✓ Windows applications
- ✓ Web applications
- ✓ Web services

The .Net framework consists of an enormous library of codes used by the client languages like C#. Following are some of the components of the .Net framework:

- Common Language Runtime (CLR)
- The .Net Framework Class Library
- Common Language Specification (CLS)
- Common Type System (CTS)
- Metadata and Assemblies
- Windows Forms
- ASP.Net and ASP.Net AJAX
- ADO.Net
- Windows Workflow Foundation (WF)
- Windows Presentation Foundation
- Windows Communication Foundation (WCF)
- LINQ

3.1.1.2. Integrated Development Environment (IDE) For C# :

Microsoft provides the following development tools for C# programming:

- ✓ Visual Studio 2010 (VS)
- ✓ Visual C# 2010 Express (VCE)
- ✓ Visual Web Developer

3.1.1.3. C# Program Structure:

Before we study basic building blocks of the C# programming language, let us look at a bare minimum C# program structure.

C# Hello World Example

A C# program basically consists of the following parts:

Namespace declaration
A class
Class methods
Class attributes
A Main method
Statements & Expressions
Comments

Example:

```
using System;
namespace HelloWorldApplication
{
    class HelloWorld
    {
        static void Main(string[] args)
        {
            /* my first program in C# */
            Console.WriteLine("Hello World");
            Console.ReadLine();
        }
    }
}
```

- The first line of the program using System; - the using keyword is used to include the System namespace in the program. A program generally has multiple using statements.
- The next line has the namespace declaration. A namespace is a collection of classes. The HelloWorldApplication namespace contains the class HelloWorld.
- The next line has a class declaration, the class HelloWorld, contains the data and method definitions that the program uses. Classes generally would contain more than one method. Methods define the behavior of the class.
- The next line defines the Main method, which is the entry point for all C# programs. The Main method states what the class will do when executed.
- The next line /*...*/ will be ignored by the compiler and it has been put to add additional comments in the program.
- The Main method specifies its behavior with the statement Console.WriteLine("Hello World");
- WriteLine is a method of the Console class defined in the System namespace. This statement causes the message "Hello, World!" to be displayed on the screen.
- The last line Console.ReadLine(); makes the program wait for a key press and it prevents the screen from running and closing quickly when the program is launched from Visual Studio .NET.

NOTE:

- C# is case sensitive.
- All statements and expression must end with a semicolon (;).
- The program execution starts at the Main method.
- Unlike Java, file name could be different from the class name.

Compilation

- Type **csc helloworld.cs** and press enter to compile the code.
- If there are no errors in our code the command prompt will take us to the next line and would generate helloworld.exe executable file.

3.1.2. Data Types:

In C#, Data Types are categorized into the following types:

1. Value Types
2. Reference Types
3. Pointer Types

1. Value Types:

- A data type is a value type if it holds a data value within its own memory space. It means variables of these data types directly contain their values.
- All the value types derive from System.ValueType, which in-turn, derives from System.Object.
- For example, int, char, bool, etc.

Type	Represents	Range	Default Value
bool	Boolean value	True or False	False
byte	8-bit unsigned integer	0 to 255	0
char	16-bit Unicode character	U +0000 to U +ffff	'\0'
decimal	128-bit precise decimal values with 28-29 significant digits	$(-7.9 \times 10^{28} \text{ to } 7.9 \times 10^{28}) / 10^{0 \text{ to } 28}$	0.0M
double	64-bit double-precision floating point type	$(+/-)5.0 \times 10^{-324} \text{ to } (+/-)1.7 \times 10^{308}$	0.0D
float	32-bit single-precision floating point type	$-3.4 \times 10^{38} \text{ to } +3.4 \times 10^{38}$	0.0F
int	32-bit signed integer type	-2,147,483,648 to 2,147,483,647	0
long	64-bit signed integer type	-923,372,036,854,775,808 to 9,223,372,036,854,775,807	0L
sbyte	8-bit signed integer type	-128 to 127	0
short	16-bit signed integer type	-32,768 to 32,767	0
uint	32-bit unsigned integer type	0 to 4,294,967,295	0
ulong	64-bit unsigned integer type	0 to 18,446,744,073,709,551,615	0
ushort	16-bit unsigned integer type	0 to 65,535	0

To get the exact size of a type or a variable on a particular platform, we can use the sizeof method. The expression sizeof(type) yields the storage size of the object or type in bytes.

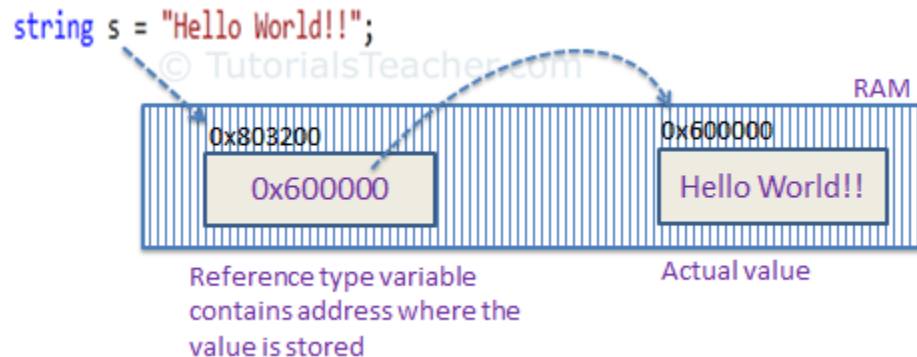
2. Reference Types:

- A reference type are such data types that stores the address instead of value.
- In other words, a reference type contains a pointer to another memory location that holds the data.

For example, consider following string variable:

```
string s = "Hello World!!";
```

The following image shows how the system allocates the memory for the above string variable.



Memory allocation for Reference type

As you can see in the above image, the system selects a random location in memory (0x803200) for the variable 's'. The value of a variable s is 0x600000 which is the memory address of the actual data value. Thus, reference type stores the address of the location where the actual value is stored instead of value itself.

The following data types are of reference type:

- String
- All arrays, even if their elements are value types
- Class
- Delegates

3. Pointer Types:

- Pointer type variables store the memory address of another type. Pointers in C# have the same capabilities as the pointers in C or C++.

Syntax for declaring a pointer type is:

type* identifier;

For example:

```
char* cptr;  
int* iptr;
```

3.1.3. C# Type Conversion/Type casting:

Type conversion is basically type casting, or converting one type of data to another type. In C#, type casting has two forms:

1. **Implicit type conversion** - these conversions are performed by C# in a type-safe manner. Examples are conversions from smaller to larger integral types, and conversions from derived classes to base classes.
2. **Explicit type conversion** - users using the pre-defined functions do these conversions explicitly. Explicit conversions require a cast operator.

The following example shows an explicit type conversion:

```
namespace TypeConversionApplication {  
class ExplicitConversion  
{  
static void Main(string[] args)  
{  
double d = 5673.74;  
int i;  
cast double to int. i = (int)d; Console.WriteLine(i); Console.ReadLine();  
}  
}  
}
```

Output:

5673

C# Type Conversion Methods:

C# provides the following built-in type conversion methods:

1. **ToBoolean**: Converts a type to a Boolean value, where possible.
2. **ToByte**: Converts a type to a byte.
3. **ToChar**: Converts a type to a single Unicode character, where possible.
4. **ToDateTime**: Converts a type (integer or string type) to date-time structures.
5. **ToDecimal**: Converts a floating point or integer type to a decimal type.
6. **ToDouble**: Converts a type to a double type.
7. **ToInt16**: Converts a type to a 16-bit integer.
8. **ToInt32**: Converts a type to a 32-bit integer.
9. **ToInt64**: Converts a type to a 64-bit integer.
10. **ToSbyte**: Converts a type to a signed byte type.
11. **ToSingle**: Converts a type to a small floating-point number.
12. : Converts a type to a string.
13. : Converts a type to a specified type.
14. **ToUInt16**: Converts a type to an unsigned int type.
15. **ToUInt32**: Converts a type to an unsigned long type.
16. **ToUInt64**: Converts a type to an unsigned big integer.

3.1.4. C# Variable:

- The variable is a name given to a data value or memory storage that program can manipulate.
- A variable holds the value of specific data type e.g. string, int, float etc.
- A variable can be declared and initialized in separate statements and also in the single statement.
- The value of a variable can be changed at any time throughout out the program as long as it is accessible.

Declaration of Variable:**Syntax:**

<data type> <variable name>;

Example: int j; char c;

Initialization of Variable:**Syntax:**

<datatype> <variable name> = <value>;

Example: int j=10; String c=" Hello";

3.1.5. C# Constant/Literals:

- The constants refer to fixed values that the program may not alter during its execution. These fixed values are also called literals.
- Constants can be of any of the basic data types like an integer constant, a floating constant, a character constant, or a string literal. There are also enumeration constants as well.
- The constants are treated just like regular variables except that their values cannot be modified after their definition.

Defining a constant or constant Declaration and initialization:

const <data_type> <constant_name> = value; **Example:** const double pi = 3.14159;

❖ Literals types:

There are following 4 types of literals. They are described below:

- a) Integer Literals
- b) Floating point Literals
- c) Character Literals
- d) String Literals

a) Integer Literals:

- An integer literal can be a decimal, octal, or hexadecimal constant.
- A prefix specifies the base or radix: 0x or 0X for hexadecimal, 0 for octal, and nothing for an integer literal can also have a suffix that is a combination of U and L, for unsigned and long, respectively.
- The suffix can be uppercase or lowercase and can be in any order.
- *85 /* decimal */*
- *0213 /* octal */*
- *0x4b /* hexadecimal */*

b) Floating Point Literals:

- A floating-point literal has an integer part, a decimal point, a fractional part, and an exponent part.
- We can represent floating point literals either in decimal form or exponential form.
- Here are some examples of floating-point literals:
3.14159 /* Legal */
314159E-5L /* Legal */
510E /* Illegal: incomplete exponent */
210f /* Illegal: no decimal or exponent */
.e55 /* Illegal: missing integer or fraction */

While representing using decimal form, we must include the decimal point, the exponent, or both and while representing using exponential form we must include the integer part, the fractional part, or both. The signed exponent is introduced by e or E.

c) Character Literals:

- Character literals are enclosed in single quotes e.g., 'x' and can be stored in a simple variable of char type.
- A character literal can be a plain character (e.g., 'x'), an escape sequence (e.g., '\t'), or a universal character (e.g., '\u02C0').
- There are certain characters in C# when they are preceded by a back slash they will have special meaning and they are used to represent like newline (\n) or tab (\t).
- Here you have a list of some of such escape sequence codes:

<u>Escape sequence</u>	<u>Meaning</u>
\\	\ character
'	' character
\"	" character
\?	? character
\a	Alert or bell
\b	Backspace
\f	Form feed
\n	Newline
\r	Carriage return
\t	Horizontal tab
\v	Vertical tab
\ooo	Octal number of one to three digits
\xhh . . .	Hexadecimal number of one or more digits

d) String Literals:

- String literals or constants are enclosed in double quotes "" or with @"".
- A string contains characters that are similar to character literals: plain characters, escape sequences, and universal characters.
- Here are some examples of string literals. All the three forms are identical strings.
"hello, dear"
@"hello dear"

3.1.6. Identifier:

- An identifier is a name used to identify a class, variable, function, or any other user-defined item.
- The basic rules for naming classes in C# are as follows:
 - ✓ A name must begin with a letter that could be followed by a sequence of letters, digits (0 - 9) or underscore.
 - ✓ The first character in an identifier cannot be a digit.
 - ✓ It must not contain any embedded space or symbol like ? - + ! @ # % ^ & * () [] { } . ; : " ' / and \. However an underscore (_) can be used.
 - ✓ It should not be a C# keyword.

3.1.7. C# Keywords:

- Keywords are reserved words predefined to the C# compiler.
- These keywords cannot be used as identifiers, however, if we want to use these keywords as identifiers, we may prefix the keyword with the @ character.
- In C# some identifiers have special meaning in context of code, such as get and set, these are called contextual keywords.

The following table lists the reserved keywords in C#:

abstract	As	Base	bool	break	byte	case
catch	Char	checked	class	const	continue	decimal
default	delegate	do	double	else	enum	event
explicit	extern	false	finally	fixed	float	for
foreach	Goto	if	implicit	in	in	int
interface	internal	is	lock	long	namespae	new
null	object	operator	out	out	override	params
private	protected	public	readonly	ref	return	sbyte
sealed	short	sizeof	stackalloc	static	string	struct
switch	This	throw	true	try	typeof	uint
ulong	unchecked	unsafe	ushort	using	virtual	void
volatile			While			

3.2. C# statements, objects and classes, array and strings:**3.2.1. C# Statements:**

There are three types of statements. They are Decision making statements, Looping statement and Loop Control Statement.

1. Decision making or Conditional Statements:

Following are the Decision-making statements available in C#. They are as follows:

- if statement
- if-else statement
- switch statement
- Ternary operator:?

a. If Statement:

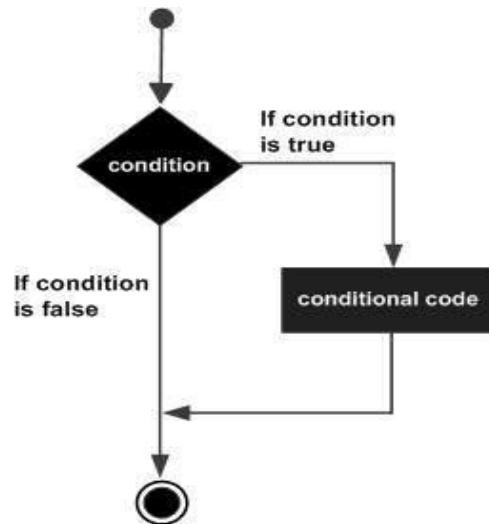
- An if statement consists of a boolean expression followed by one or more statements.

Syntax:

The syntax of an if statement in C# is:

```
if(boolean_expression)
{
    /* statement(s) will execute if the boolean expression is true */
}
```

If the boolean expression evaluates to true, then the block of code inside the if statement is executed. If boolean expression evaluates to false, then the first set of code after the end of the if statement (after the closing curly brace) is executed.

Flow Diagram:

Example: if condition

```
int i = 10, j = 20;
if (i > j)
{
    Console.WriteLine("i is greater than j");
}
if (i < j)
{
    Console.WriteLine("i is less than j");
}
if (i == j)
{
    Console.WriteLine("i is equal to j");
}
```

Output:

i is less than j

b. if-else Statement:

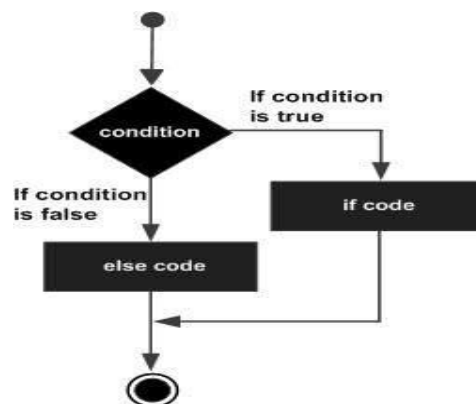
- An if statement can be followed by an optional else statement, which executes when the boolean expression is false.

Syntax:

The syntax of an if...else statement in C# is :

```
if(boolean_expression) {  
    /* statement(s) will execute if the boolean expression is true */  
} else {  
    /* statement(s) will execute if the boolean expression is false */  
}
```

If the boolean expression evaluates to true, then the if block of code is executed, otherwise else block of code is executed.

Flow Diagram**Example:**

```
using System;  
namespace DecisionMaking {  
    class Program {  
        static void Main(string[] args) {  
            int a = 100;  
            if (a < 20) {  
                Console.WriteLine("a is less than 20");  
            } else {  
                Console.WriteLine("a is not less than 20");  
            }  
            Console.WriteLine("value of a is : {0}", a);  
            Console.ReadLine();  
        }  
    }  
}
```

When the above code is compiled and executed, it produces the following result :

```
a is not less than 20;  
value of a is: 100
```

❖ Nested If Statements:

- It is always legal in C# to nest if-else statements, which means you can use one if or else if statement inside another if or else if statement(s).

Syntax:

The syntax for a nested if statement is as follows:

```
if( boolean_expression 1)
{
    /* Executes when the boolean expression 1 is true */
    if(boolean_expression 2)
    {
        /* Executes when the boolean expression 2 is true */
    }
}
```

You can nest else if...else in the similar way as you have nested if statement.

Example:

```
using System;
namespace DecisionMaking {
    class Program {
        static void Main(string[] args) {
            /* local variable definition */
            int a = 100;
            int b = 200;
            /* check the boolean condition */
            if (a == 100) {
                /* if condition is true then check the following */
                if (b == 200) {
                    /* if condition is true then print the following */
                    Console.WriteLine("Value of a is 100 and b is 200");
                }
            }
            Console.WriteLine("Exact value of a is : {0}", a);
            Console.WriteLine("Exact value of b is : {0}", b);
            Console.ReadLine();
        }
    }
}
```

Output

When the above code is compiled and executed, it produces the following result:

```
Value of a is 100 and b is 200
Exact value of a is : 100
Exact value of b is : 200
```

Example: else if

```
static void Main(string[] args)
{
    int i = 10, j = 20;
    if (i > j)
    {
        Console.WriteLine("i is greater than j");
    }
    else if (i < j)
    {
        Console.WriteLine("i is less than j");
    }
    else
    {
        Console.WriteLine("i is equal to j");
    }
}
```

Output:

i is less than j

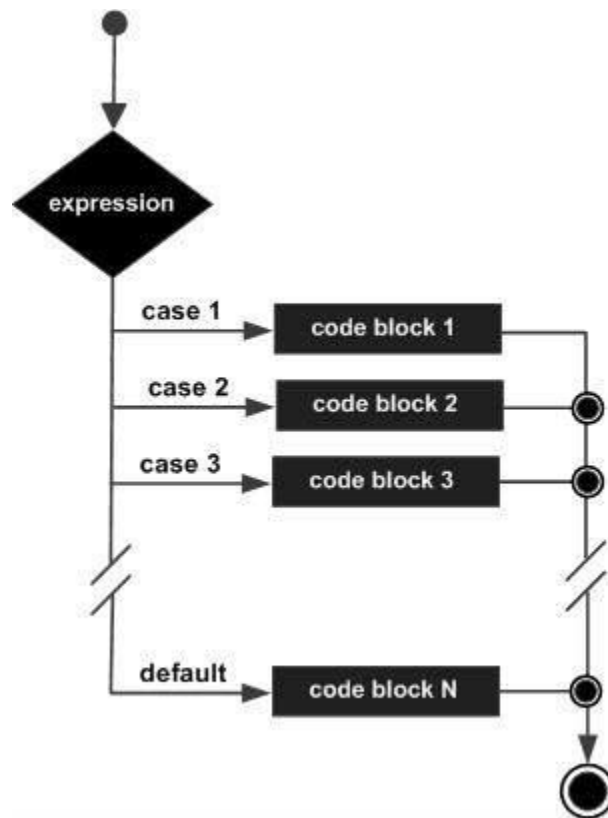
c. Switch Statement:

- A switch statement allows a variable to be tested for equality against a list of values.
- Each value is called a case, and the variable being switched on is checked for each switch case.

Syntax:

The syntax for a switch statement in C# is as follows:

```
switch(expression)
{
    case <value1>
        // code block
        break;
    case <value2>
        // code block
        break;
    case <valueN>
        // code block
        break;
    default
        // code block
        break;
}
```

Flow Diagram:**Example:**

```
using System;
namespace DecisionMaking {
    class Program {
        static void Main(string[] args) {
            /* local variable definition */
            char grade = 'B';
            switch (grade) {
                case 'A':
                    Console.WriteLine("Excellent!");
                    break;
                case 'B':
                case 'C':
                    Console.WriteLine("Well done");
                    break;
                case 'D':
                    Console.WriteLine("You passed");
                    break;
                case 'F':
                    Console.WriteLine("Better try again");
                    break;
                default:
                    Console.WriteLine("Invalid grade");
            }
        }
    }
}
```

```
        break;
    }
    Console.WriteLine("Your grade is {0}", grade);
    Console.ReadLine();
}
}
```

Output:

When the above code is compiled and executed, it produces the following result:

Well done

Your grade is B

d. Ternary Operator (?):

- C# includes a special type of decision making operator '?' called the ternary operator.
- The ternary operator can return a value of any data type.

Syntax:

Boolean Expression ? First Statement : Second Statement

- Ternary operator returns a value or expression included in the second or third part of it. It does not execute the statements.

Example: Ternary operator

```
int x = 20, y = 10;
```

```
var result = x > y ? x : y;
```

```
Console.WriteLine(result);
```

output:

20

Ternary operator can also be used instead of if-else statement. The above example can be written using if-else statement as shown below.

Example: Ternary operator replaces if statement

```
int x = 20, y = 10;
```

```
int result = 0;
```

```
if (x > y)
```

```
    result = x;
```

```
else if (x < y)
```

```
    result = y;
```

```
Console.WriteLine(result);
```

output:

20

2. Looping Statements:

The looping statements are nothing but repetition and execution of group of statements till the condition met.

There are 3 types of looping statements.

- For Loop
- While Loop
- Do while Loop

a. For Loop:

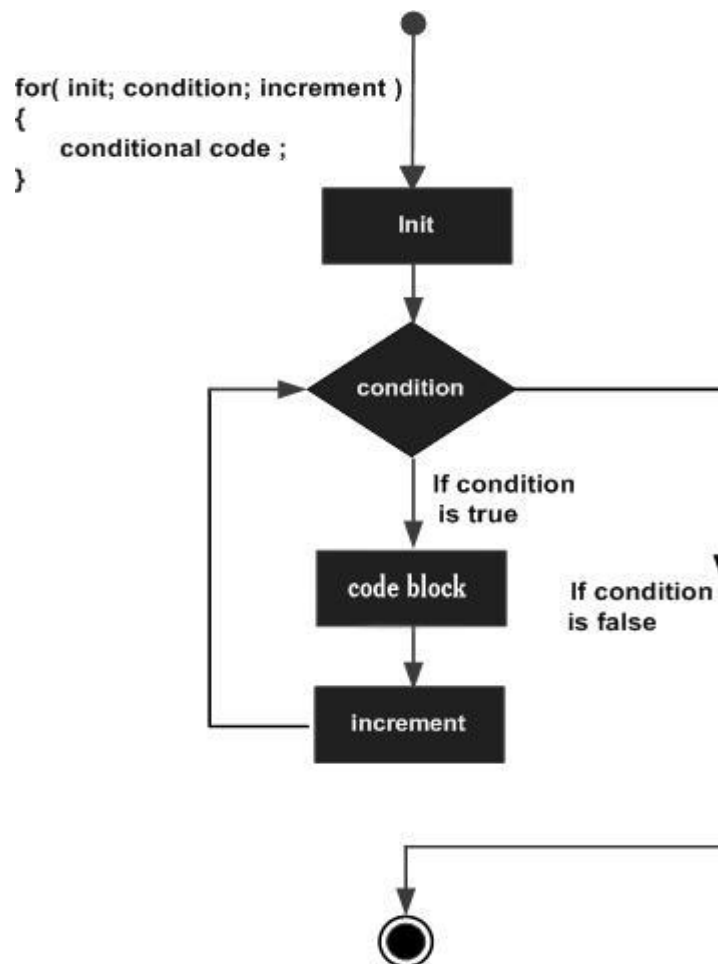
- A for loop is a repetition control structure that allows you to efficiently write a loop that needs to execute a specific number of times.

Syntax:

The syntax of a for loop in C# is :

```
for (init; condition; increment )  
{  
    statement(s);  
}
```

Flow Diagram



Example:

```
using System;
namespace Loops {
    class Program {
        static void Main(string[] args) {
            /* for loop execution */
            for (int a = 10; a < 20; a = a + 1) {
                Console.WriteLine("value of a: {0}", a);
            }
            Console.ReadLine();
        }
    }
}
```

Output:

```
value of a: 10
value of a: 11
value of a: 12
value of a: 13
value of a: 14
value of a: 15
value of a: 16
value of a: 17
value of a: 18
value of a: 19
```

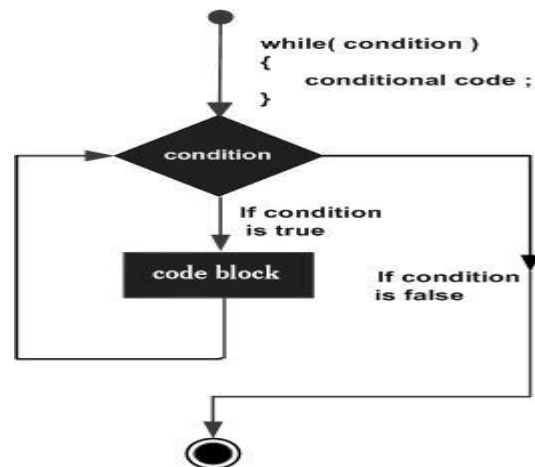
b. While Loop:

- A while loop statement in C# repeatedly executes a target statement as long as a given condition is true.

Syntax:

The syntax of a while loop in C# is:

```
while(condition)
{
    statement(s);
}
```

Flow Diagram

Here, key point of the while loop is that the loop might not ever run. When the condition is tested and the result is false, the loop body is skipped and the first statement after the while loop is executed.

Example

```
using System;
namespace Loops {
    class Program {
        static void Main(string[] args) {
            /* local variable definition */
            int a = 10;
            /* while loop execution */
            while (a < 20) {
                Console.WriteLine("value of a: {0}", a);
                a++;
            }
            Console.ReadLine();
        }
    }
}
```

Output:

```
value of a: 10
value of a: 11
value of a: 12
value of a: 13
value of a: 14
value of a: 15
value of a: 16
value of a: 17
value of a: 18
value of a: 19
```

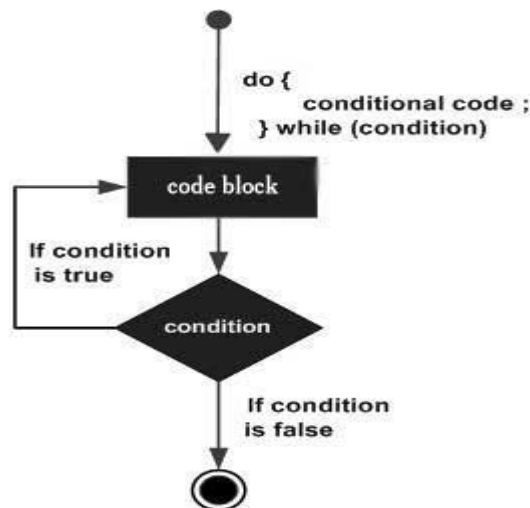
c. Do While Loop:

- Unlike for and while loops, which test the loop condition at the start of the loop, the do...while loop checks its condition at the end of the loop.
- A do...while loop is similar to a while loop, except that a do...while loop is guaranteed to execute at least one time.

Syntax:

The syntax of a do...while loop in C# is:

```
do
{
    statement(s);
}
while (condition);
```

Flow Diagram:**Example:**

```
using System;
namespace Loops {
    class Program {
        static void Main(string[] args) {
            /* local variable definition */
            int a = 10;

            /* do loop execution */
            do {
                Console.WriteLine("value of a: {0}", a);
                a = a + 1;
            }
            while (a < 20);
            Console.ReadLine();
        }
    }
}
```

Output:

value of a: 10
value of a: 11
value of a: 12
value of a: 13
value of a: 14
value of a: 15
value of a: 16
value of a: 17
value of a: 18
value of a: 19

3. Loop Control Statement:

- Loop control statements change execution from its normal sequence.
- When execution leaves a scope, all automatic objects that were created in that scope are destroyed.
- C# provides the following control statements.

a. break statement:

The break statement in C# has following two usage:

- When the break statement is encountered inside a loop, the loop is immediately terminated and program control resumes at the next statement following the loop.
- It can be used to terminate a case in the switch statement.
- If we are using nested loops (i.e. one loop inside another loop), the break statement will stop the execution of the innermost loop and start executing the next line of code after the block.

Syntax:

The syntax for a break statement in C# is as follows:

```
break;
```

Example:

```
class Test
{
    static void Main(string[] args)
    {
        for(int n=1; n <= 5 ; n++)
        {
            Console.WriteLine("*");
            if(n==2)
                break;
        }
    }
}
```

b. continue statement:

- The continue statement in C# works somewhat like the break statement.
- Instead of forcing termination, however, continue forces the next iteration of the loop to take place, skipping any code in between.
- For the for loop, continue statement causes the conditional test and increment portions of the loop to execute.
- For the while and do...while loops, continue statement causes the program control passes to the conditional tests.

Syntax:

The syntax for a continue statement in C# is as follows:

continue;

Example:

```
for (int i = 0; i < 10; i++)  
{  
    if (i == 4)  
    {  
        continue;  
    }  
    Console.WriteLine(i);  
}
```

Output

0
1
2
3
5
6
7
8
9

3.2.2. Object and Class in C#:**1. Object:**

- An object is a real entity exist in this world.
- A program may create many objects of the same class.
- Objects are also called instances, and they can be stored in either a named variable or in an array or collection.

Example: Apple is an object of a class fruit because it has all the properties that class fruit has.

2. Class:

- A class is like a blueprint of a specific object. In the real world, every object has some color, shape, and functionalities.
- Example, the luxury car Ferrari. Ferrari is an object of the luxury car type.
- The luxury car is a class that indicates some characteristics like speed, color, shape, interior, etc.
- Likewise, in object-oriented programming, a class defines some properties, fields, events, methods, etc.
- A class defines the kinds of data and the functionality their objects will have.
- A class enables you to create your custom types by grouping variables of other types, methods, and events.
- In C#, a class can be defined by using the class keyword.

Example: C# Class

```
public class MyClass
{
    public string myField = string.Empty;
    public MyClass()
    {
    }
    public void MyMethod(int parameter1, string parameter2)
    {
        Console.WriteLine("First Parameter {0}, second parameter {1}", parameter1, parameter2);
    }
    public int MyAutoImplementedProperty {get; set;}
    private int myPropertyVar;
    public int MyProperty
    {
        get { return myPropertyVar; }
        set { myPropertyVar = value; }
    }
}
```

The following image shows the important building blocks of C# class.

The image shows a C# code snippet for a class named `MyClass`. Annotations with arrows point to various parts of the code:

- Access Modifier:** Points to `public` at the start of the class declaration.
- Class name:** Points to `MyClass`.
- Field:** Points to `myField` in the line `public string myField = string.Empty;`.
- Constructor:** Points to `MyClass()` in the line `public MyClass()`.
- Method\Function:** Points to `MyMethod` in the line `public void MyMethod(int parameter1, string parameter2)`.
- Auto-implemented property:** Points to `MyAutoImplementedProperty` in the line `public int MyAutoImplementedProperty { get; set; }`.
- Property:** A bracket groups the `private int myPropertyVar;` line and the `MyProperty` property block, with the label `Property` pointing to the bracket.

The code snippet is as follows:

```

public class MyClass
{
    public string myField = string.Empty;
    public MyClass()
    {
    }
    public void MyMethod(int parameter1, string parameter2)
    {
        Console.WriteLine("First Parameter {0}, second parameter {1}", parameter1, parameter2);
    }
    public int MyAutoImplementedProperty { get; set; }
    private int myPropertyVar;
    public int MyProperty
    {
        get { return myPropertyVar; }
        set { myPropertyVar = value; }
    }
}

```

- **C# Access Modifiers:**

- ✓ Access modifiers are applied to the declaration of the class, method, properties, fields, and other members.
- ✓ They define the accessibility of the class and its members. Public, private, protected, and internal are access modifiers in C#.

- **C# Field:**

- ✓ The field is a class-level variable that holds a value. Generally, field members should have a private access modifier and used with property.

- **C# Constructor:**

- ✓ A class constructor is a special member function of a class that is executed whenever we create new objects of that class.
- ✓ A constructor has exactly the same name as that of class and it does not have any return type
- ✓ Constructors can be defined by using an access modifier and class name:
`<access modifiers> <class name> () { }`

Example: Constructor in C#

```
// Create a Car class
class Car
{
    public string model; // Create a field
    // Create a class constructor for the Car class
    public Car()
    {
        model = "Mustang"; // Set the initial value for model
    }
    static void Main(string[] args)
    {
        Car Ford = new Car(); // Create an object of the Car Class (this will call the constructor)
        Console.WriteLine(Ford.model); // Print the value of model
    }
}
// Output:
"Mustang"
```

- **C# Method:**

- ✓ A method can be defined using the following template:
- ✓

{access modifier} {return type} MethodName({parameterType parameterName})

Example: Method in C#

```
public void MyMethod(int parameter1, string parameter2)
{
    // write your method code here..
}
```

- **Property:**

- ✓ A property can be defined using getters and setters, as shown below:

Example: Property in C#

```
private int _myPropertyVar;
public int MyProperty
{
    get { return _myPropertyVar; }
    set { _myPropertyVar = value; }
}
```

Property encapsulates a private field. It provides getters (get{ }) to retrieve the value of the underlying field and setters (set{ }) to set the value of the underlying field.

- **Namespace:**

- ✓ The namespace is a container for a set of related classes and namespaces.
- ✓ The namespace is also used to give unique names to classes within the namespace name.
- ✓ Namespace and classes are represented using a dot (.).
- ✓ In C#, namespace can be defined using the namespace keyword.

Example: Namespace

```
namespace CSharpTutorials
{
    class MyClass
    {
    }
}
```

In the above example, the fully qualified class name of MyClass is CSharpTutorials.MyClass. A namespace can contain other namespaces. Inner namespaces can be separated using (.).

3.2.3. Array:

- An array is a type of data structure that stores a fixed-size sequential collection of elements of the same data type.
- All arrays consist of contiguous memory locations.
- The lowest address corresponds to the first element and the highest address to the last element.

**❖ Declaring Arrays:**

To declare an array in C#, you can use the following syntax:

```
datatype[] arrayName;
```

where,

1. datatype is used to specify the type of elements to be stored in the array.
2. [] specifies the rank of the array. The rank specifies the size of the array.
3. arrayName specifies the name of the array.

For example,

```
int[] nums; // stores integer values
string[] names; // stores string values
double[] salaries; // stores double values
```

❖ Initializing and assigning values to an Array:

- Declaring an array does not initialize the array in the memory.
- When the array variable is initialized, we can assign values to the array.
- Array is a reference type, so we need to use the new keyword to create an instance of the array.

```
double[] balance = new double[10]; double[] balance = new double[10]; balance[0] = 4500.0;
```

- We can assign values to the array at the time of declaration, like:
`double[] balance = { 2340.0, 4523.69, 3421.0};`
- We can also create and initialize an array, like:
`int [] marks = new int[5] { 99, 98, 92, 97, 95};`
- In the preceding case, we may also omit the size of the array, like:
`int [] marks = new int[] { 99, 98, 92, 97, 95};`
- We can also copy an array variable into another target array variable. In that case, both the target and source would point to the same memory location:
`int [] marks = new int[] { 99, 98, 92, 97, 95}; int[] score = marks;`

❖ Accessing an Array Elements:

- Indexing the array name accesses an element.
- Placing the index of the element within square brackets after the name of the array does this.

❖ For example:

```
double salary = balance[9];
```

❖ Types of Array:

- There are three types array .They are described below:
 - 1.Single-Dimensional Array
 2. Multi-Dimensional Array
 3. Jagged Array

1. Single -Dimensional Array:

- The one-dimensional array or single dimensional array in C# is the simplest type of array that contains only one row or column for storing data.
- It has single set of square brackets (“[]”).
- To declare single dimensional array in C#,

```
datatype[] arrayName;
```

Example:

```
string[] Books = new string[5];
```

The array age is a one-dimensional array that contains only 5 elements in a single row.

Programming example of one-dimensional array:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace One_Dimensional_Array
{
    class Program
    {
        static void Main(string[] args)
        {
            //Declaring single dimensional array
            string[] Books = new string[5];
            Books[0] = "C#";
            Books[1] = "Java";
            Books[2] = "VB.NET";
            Books[3] = "C++";
            Books[4] = "C";

            Console.WriteLine("All the element of Books array is:\n\n");
            int i = 0;
            //Formatting Output
            Console.Write("\t1\t2\t3\t4\t5\n\n\t");
            for (i = 0; i < 5; i++)
            {
                Console.Write("{0}\t", Books[i]);
            }
            Console.ReadLine();
        }
    }
}
```

Output

All the element of Books array is:

1	2	3	4	5
C#	Java	VB.NET	C++	C

2. Multi-Dimensional Array:

- A two-dimensional array can be thought of as a table which will have x number of rows and y number of columns.
- The multidimensional array can be declared by adding commas in the square brackets.
- For example, [,] declares two-dimensional array, [, ,] declares three-dimensional array, [, , ,] declares four-dimensional array, and so on.
- So, in a multidimensional array, no of commas = No of Dimensions - 1.

The following declares multidimensional arrays.

Example: Multidimensional Arrays

```
int[,] arr2d; // two-dimensional array
int[, ,] arr3d; // three-dimensional array
int[, , ,] arr4d ; // four-dimensional array
int[, , , ,] arr5d; // five-dimensional array
```

❖ Accessing Two-Dimensional Array Elements

- Using the subscripts i.e. accesses an element in 2-dimensional array. row index and column index of the array.
- For example:

```
int val = a[2,3];
```

The above statement will take 4th element from the 3rd row of the array.

3. Jagged Array:

- A jagged array is an array of array.
- Jagged arrays store arrays instead of literal values.
- A jagged array is initialized with two square brackets [[]].
- The first bracket specifies the size of an array, and the second bracket specifies the dimensions of the array which is going to be stored.

The following example declares jagged arrays.

Example: Jagged Arrays

```
int[][] jArray1 = new int[2][]; // can include two single-dimensional arrays
int[, ,] jArray2 = new int[3][,]; // can include three two-dimensional arrays
```

In the above example, jArray1 can store up to two single-dimensional arrays. jArray2 can store up to three two-dimensional, arrays [,] specifies the two-dimensional array.

Example: Jagged Array

```
class ArrayTest
{
    static void Main()
    {
        // Declare the array of two elements.
        int[][] arr = new int[2][];
        // Initialize the elements.
        arr[0] = new int[5] { 1, 3, 5, 7, 9 };
        arr[1] = new int[4] { 2, 4, 6, 8 };
        // Display the array elements.
        for (int i = 0; i < arr.Length; i++)
        {
            System.Console.Write("Element({0}): ", i);
            for (int j = 0; j < arr[i].Length; j++)
            {
                System.Console.Write("{0}{1}", arr[i][j], j == (arr[i].Length - 1) ? "" : " ");
            }
            System.Console.WriteLine();
        }
        // Keep the console window open in debug mode.
        System.Console.WriteLine("Press any key to exit.");
        System.Console.ReadKey();
    }
}
/* Output:
Element(0): 1 3 5 7 9
Element(1): 2 4 6 8
*/
```

3.2.4. Strings in C#:

- Array of character is string.
- Strings are used for storing text.
- A string variable contains a collection of characters surrounded by double quotes:

❖ Declaring and initializing a string:

Declaration without size:

Syntax:

String[] variable_name;

or

string[] variable_name;

Declaration with size:

Syntax:

String[] variable_name = new String[provide_size_here];

or

string[] variable_name = new string[provide_size_here];

Initializing Array of String:

- Initialization without giving size is not valid in C#. It will give compile time error.

// Declaration of the array

```
string[] str1, str2;
```

// Initialization of array

```
str1 = new string[5]{ "Element 1", "Element 2", "Element 3", "Element 4", "Element 5" };
```

or

```
str2 = new string[] { "Element 1", "Element 2", "Element 3", "Element 4", "Element 5" };
```

// C# program to illustrate the String array

// declaration, initialization and accessing its elements

```
using System;
```

```
class Geeks {
```

```
    // Main Method
```

```
    public static void Main()
```

```
    {
```

```
        // Step 1: Array Declaration
```

```
        string[] stringarr;
```

```
        // Step 2: Array Initialization
```

```
        stringarr = new string[3] { "Element 1", "Element 2", "Element 3" };
```

```
        // Step 3: Accessing Array Elements
```

```
        Console.WriteLine(stringarr[0]);
```

```
        Console.WriteLine(stringarr[1]);
```

```
        Console.WriteLine(stringarr[2]);
```

```
    } }
```

Output:

Element 1

Element 2

Element 3

❖ Member Functions and Encapsulation:

A member function of a class is a function that has its definition or its prototype within the class definition like any other variable. It operates on any object of the class of which it is a member, and has access to all the members of a class for that object.

Member variables are attributes of an object (from design perspective) and they are kept private to implement encapsulation. These variables can only be accessed using the public member functions. The following example program can demonstrate this concept.

Example:

```
using System;
namespace BoxApplication
{
    class Box
    {
        private double length;    // Length of a box private double breadth; // Breadth of a box private
        double height;           // Height of a box public void setLength( double len )
        {
            length = len;
        }
        public void setBreadth( double bre )
        {
            breadth = bre;
        }
        public void setHeight( double hei )
        {
            height = hei;
        }
        public double getVolume()
        {
            return length * breadth * height;
        }
    }
    class Boxtester
    {
        static void Main(string[] args)
        {
            Box Box1 = new Box(); // Declare Box1 of type Box Box Box2 = new Box();// Declare Box2 of
            type Box double volume;
            box 1 specification Box1.setLength(6.0); Box1.setBreadth(7.0); Box1.setHeight(5.0);
            box 2 specification Box2.setLength(12.0); Box2.setBreadth(13.0); Box2.setHeight(10.0);
            //volume of box 1
            volume = Box1.getVolume(); Console.WriteLine("Volume of Box1 : {0}" ,volume);
            // volume of box 2
            volume = Box2.getVolume(); Console.WriteLine("Volume of Box2 : {0}", volume);
            Console.ReadLine();
        }
    }
}
```

Output:

Volume of Box1 : 210
Volume of Box2 : 1560

3.3. System Collection, Delegates and Events, Indexes Attributes, Versioning:**3.3.1. System Collection/C# Collections:**

- C# also includes specialized classes that hold many values or objects in a specific series, that are called 'collection'.
- Collection classes are specialized classes for data storage and retrieval.
- These classes provide support for stacks, queues, lists, and hash tables
- There are two types of collections available in C#: non-generic collections and generic collections.
- Collection classes serve various purposes, such as allocating memory dynamically to elements and accessing a list of items on the basis of an index etc.

❖ Various Collection Classes and Their Usage

The following are the various commonly used classes of the **System.Collection** namespace

1. ArrayList:

- ✓ It represents ordered collection of an object that can be indexed individually.
- ✓ It is basically an alternative to an array.
- ✓ ArrayList stores objects of any type like an array. However, there is no need to specify the size of the ArrayList like with an array as it grows automatically.
- ✓ It also allows dynamic memory allocation, add, search and sort items in the list.

2. Hashtable:

- ✓ It uses a key to access the elements in the collection.
- ✓ A hash table is used when we need to access elements by using key, and we can identify a useful key value. Each item in the hash table has a key/value pair.
- ✓ The key is used to access the items in the collection.

3. Sortedlist:

- ✓ It uses a key as well as an index to access the items in a list.
- ✓ A sorted list is a combination of an array and a hash table.
- ✓ It automatically arranges elements in ascending order of key by default.
- ✓ C# includes both, generic and non-generic SortedList collection.

4. Stack:

- ✓ Stack stores the values in LIFO style (Last In First Out).
- ✓ It provides a Push() method to add a value and Pop() & Peek() methods to retrieve values.
- ✓ C# includes both, generic and non-generic Stack.
- ✓ It represents a last-in, first out collection of object.

5. Queue:

- ✓ Queue stores the values in FIFO style (First In First Out).
- ✓ It keeps the order in which the values were added.
- ✓ It provides an Enqueue() method to add values and a Dequeue() method to retrieve values from the collection.
- ✓ C# includes generic and non-generic Queue.

6. BitArray:

- ✓ It represents an array of the binary representation using the values 1 and 0.
- ✓ It is used when we need to store the bits but do not know the number of bits in advance.
- ✓ We can access items from the BitArray collection by using an integer index, which starts from zero.

3.3.2. Delegates and Events:

- A delegate is like a pointer to a function. It is a reference type data type and it holds the reference of a method.
- All the delegates are implicitly derived from System.Delegate class.
- A delegate can be declared using delegate keyword followed by a function signature as shown below.

Delegate Syntax:

<access modifier> delegate <return type> <delegate_name>(<parameters>)

The following example declares a Print delegate.

Example: Declare delegate

```
public delegate void Print(int value);
```

The Print delegate shown above, can be used to point to any method that has same return type & parameters declared with Print.

Consider the following example that declares and uses Print delegate.

Example: C# delegate

```
class Program
{
    // declare delegate
    public delegate void Print(int value);
    static void Main(string[] args)
    {
        // Print delegate points to PrintNumber
        Print printDel = PrintNumber;
        // or
        // Print printDel = new Print(PrintNumber);
        printDel(100000);
        printDel(200);
        // Print delegate points to PrintMoney
        printDel = PrintMoney;
        printDel(10000);
        printDel(200);
    }
    public static void PrintNumber(int num)
    {
        Console.WriteLine("Number: {0,-12:N0}",num);
    }
}
```

```
public static void PrintMoney(int money)
{
    Console.WriteLine("Money: {0:C}", money);
}
}
```

Output:

Number: 10,000

Number: 200

Money: \$ 10,000.00

Money: \$ 200.00

❖ Events:

- An event is nothing but an encapsulated delegate.
- To declare an event, use the event keyword before declaring a variable of delegate type
- Events are user actions such as key press, clicks, mouse movements, etc., or some occurrence such as system generated notifications.
- Applications need to respond to events when they occur. For example, interrupts. Events are used for inter-process communication.

Example: Event Declaration

```
public delegate void someEvent();
```

```
public event someEvent someEvent;
```

Thus, a delegate becomes an event using the event keyword.

Example:

```
namespace SampleApp {
    public delegate string MyDel(string str);
    class EventProgram {
        event MyDel MyEvent;
        public EventProgram() {
            this.MyEvent += new MyDel(this.WelcomeUser);
        }
        public string WelcomeUser(string username) {
            return "Welcome " + username;
        }
        static void Main(string[] args) {
            EventProgram obj1 = new EventProgram();
            string result = obj1.MyEvent("Class");
            Console.WriteLine(result);
        }
    }
}
```

Output:

Welcome Class

3.3.3. Indexer:

- An Indexer is a special type of property that allows a class or structure to be accessed like an array for its private collection.
- An indexer can be defined the same way as property with **this** keyword and square brackets [].

Syntax:

```
<return type> this[<parameter type> index]
{
    get{
        // return the value from the specified index of an internal collection
    }
    set{
        // set values at the specified index in an internal collection
    }
}
```

Example:

```
using System;
public class Program
{
    public static void Main()
    {
        StringDataStore strStore = new StringDataStore();
        strStore[0] = "one";
        strStore[1] = "Two";
        strStore[2] = "Three";
        strStore[3] = "Four";

        for(int i = 0; i < 10 ; i++)
            Console.WriteLine(strStore[i]);
    }
}

public class StringDataStore
{
    private string[] strArr = new string[10]; // internal data storage
    public StringDataStore()
    {
    }
    public string this[int index]
    {
        get
        {
            if (index < 0 && index >= strArr.Length)
                throw new IndexOutOfRangeException("Cannot store more than 10 objects");
            return strArr[index];
        }
    }
}
```

```
}  
set  
{  
    if (index < 0 && index >= strArr.Length)  
        throw new IndexOutOfRangeException("Cannot store more than 10 objects");  
    strArr[index] = value;  
}  
}  
}
```

Output:

one
Two
Three
Four

3.3.4. C# Attributes:

- An attribute is a declarative tag that is used to convey information to runtime about the behaviors of various elements like classes, methods, structures, enumerators, assemblies etc. in your program.
- You can add declarative information to a program by using an attribute.
- A declarative tag is depicted by square ([]) brackets placed above the element it is used for.
- Attributes are used for adding metadata, such as compiler instruction and other information such as comments, description, methods and classes to a program.
- The .Net Framework provides two types of attributes: the pre-defined attributes and custom-built attributes.

Specifying an Attribute

Syntax for specifying an attribute is as follows:

```
[attribute(positional_parameters, name_parameter = value, ...)]  
element
```

Predefined Attributes

The .Net Framework provides three pre-defined attributes:

1. AttributeUsage
2. Conditional
3. Obsolete

3.3.5. Versioning:

- versioning is a naming convention applied to versions of your library to signify specific milestone events.
- Ideally, the version information, you give your library should help developers determine the compatibility with their projects that make use of older versions of that same library.

3.4. Dot Net Libraries, I/O, Namespace System, Windows Forms:

1. Dot Net Libraries:

- A library is usually a collection of classes intended for a specific purpose, like the System.Net library in .NET that is intended for internet communication
- The framework includes a large class library called Framework Class Library (FCL).

2. Namespace System:

- Namespaces in C# are used to organize too many classes so that it can be easy to handle the application.
- In a simple C# program, we use System.Console where System is the namespace and Console is the class.
- To access the class of a namespace, we need to use namespace.classname
- We can use **using** keyword so that we don't have to use complete name all the time.
- In C#, global namespace is the root namespace.
- The global: System will always refer to the namespace "System" of .Net Framework.

C# namespace example

Let's see a simple example of namespace which contains one class "Program".

```
using System;
namespace ConsoleApplication1
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello Namespace!");
        }
    }
}
```

Output:

Hello Namespace!

3. Windows Form:

- Windows Forms is a Graphical User Interface (GUI) class library which is bundled in .Net Framework.
- Its main purpose is to provide an easier interface to develop the applications for desktop, tablet, PCs. It is also termed as the WinForms.
- The applications which are developed by using Windows Forms or WinForms are known as the Windows Forms Applications that runs on the desktop computer.

3.5.Operators in C#:

An operator is a symbol that tells the compiler to perform specific mathematical or logical manipulations. C# is rich in built-in operators and provides the following type of operators:

1. Arithmetic Operators
2. Relational Operators
3. Logical Operators
4. Bitwise Operators
5. Assignment Operators
6. Misc. Operators

1. Arithmetic Operators:

- Following table shows all the arithmetic operators supported by C#.
- Assume variable A holds 10 and variable B holds 20 then:

Operator	Description	Example
+	Adds two operands	A + B will give 30
-	Subtracts second operand from the first	A - B will give -10
*	Multiply both operands	A * B will give 200
/	Divide numerator by de-numerator	B / A will give 2
%	Modulus Operator and remainder of after an integer division	B % A will give 0
++	Increment operator increases integer value by one	A++ will give 11
--	Decrement operator decreases integer value by one	A-- will give 9

2. Relational Operators:

Following table shows all the relational operators supported by C#. Assume variable A holds 10 and variable B holds 20 then:

Operator	Description	Example
==	Checks if the value of two operands is equal or not, if yes then condition becomes true.	(A == B) is not true.
!=	Checks if the value of two operands is equal or not, if values are not equal then condition becomes true.	(A != B) is true.
>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	(A > B) is not true.
<	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	(A < B) is true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	(A >= B) is not true.
<=	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	(A <= B) is true.

3. Logical Operators:

Following table shows all the logical operators supported by C#. Assume variable A holds Boolean value true and variable B holds Boolean value false then:

Operator	Description	Example
&&	Called Logical AND operator. If both the operands are non zero then condition becomes true.	(A && B) is false.
	Called Logical OR Operator. If any of the two operands is non zero then condition becomes true.	(A B) is true.
!	Called Logical NOT Operator. Use to reverses the logical state of its operand. If a condition is true then Logical NOT operator will make false.	!(A && B) is true.

4. Bitwise Operators:

- Bitwise operator works on bits and performs bit-by-bit operation. The truth tables for &, |, and ^ are as follows:

P	Q	p & q	p q	p ^ q
0	0	0	0	0
0	1	0	1	1
1	1	1	1	0
1	0	0	1	1

Assume if A = 60; and B = 13; Now in binary format they will be as follows:

A = 0011 1100

B = 0000 1101

A&B = 0000 1100

A|B = 0011 1101

A^B = 0011 0001

~A = 1100 0011

5. Assignment Operators:

Operator	Description	Example
=	Simple assignment operator, Assigns values from right side operands to left side operand	$C = A + B$ assigns value of $A + B$ into C
+=	Add AND assignment operator, It adds right operand to the left operand and assign the result to left operand	$C += A$ is equivalent to $C = C + A$
-=	Subtract AND assignment operator, It subtracts right operand from the left operand and assign the result to left operand	$C -= A$ is equivalent to $C = C - A$
*=	Multiply AND assignment operator, It multiplies right operand with the left operand and assign the result to left operand	$C *= A$ is equivalent to $C = C * A$
/=	Divide AND assignment operator, It divides left operand with the right operand and assign the result to left operand	$C /= A$ is equivalent to $C = C / A$
%=	Modulus AND assignment operator, It takes modulus using two operands and assign the result to left operand	$C \% = A$ is equivalent to $C = C \% A$
<<=	Left shift AND assignment operator	$C <<= 2$ is same as $C = C << 2$
>>=	Right shift AND assignment operator	$C >>= 2$ is same as $C = C >> 2$
&=	Bitwise AND assignment operator	$C \&= 2$ is same as $C = C \& 2$
^=	bitwise exclusive OR and assignment operator	$C \wedge= 2$ is same as $C = C \wedge 2$
=	bitwise inclusive OR and assignment operator	$C = 2$ is same as $C = C 2$

6. Miscellaneous Operators:

There are few other important operators including sizeof, typeof and ? : supported by C#.

Operator	Description	Example
sizeof()	Returns the size of a data type.	sizeof(int), will return 4.
typeof()	Returns the type of a class.	typeof(StreamReader);
&	Returns the address of an variable.	&a; will give actual address of the variable.
*	Pointer to a variable.	*a; will pointer to a variable.
? :	Conditional Expression	If Condition is true ? Then value X : Otherwise value Y
is	Determines whether an object is of a certain type.	If(Ford is Car) // checks if Ford is an object of the Car class.
As	Cast without raising an exception if the cast fails.	Object obj = new StringReader("Hello"); StreamReader r = obj as StreamReader;

Important Questions From This unit

- Q1) Explain the code snippets about connecting database in aASP.NET?
- Q2) Define Versioning, Delegate and Namespace in C#? Explain how C# solve the real world problem.
- Q3) what are Arrays in strings? How they are implemented in C#? Illustrate with example .
- Q4) What is jagged array in C#? Explain delegate with example?
- Q5) Discuss the relation between the C# and .NET framework? Explain the feature of C# ?
- Q6) Differentiate between simple array and arraylist with example?
- Q7) WAP to find the factorial of the given number using recursion function?
- Q8) What are keywords and identifiers .Explain the datatypes used in C#.
- Q9) WAP to input cities name and display them in alphabetical order.
- Q10) What are collection classes and explain about class, object , event, delegate?
- Q11) WAp to demonstrate the concept of object and class.
- Q12) WAp to print prime number between 1 to 100.
- Q13) WAP program to concatenate String.
- Q14) WAP to print the sum of digit using Function.